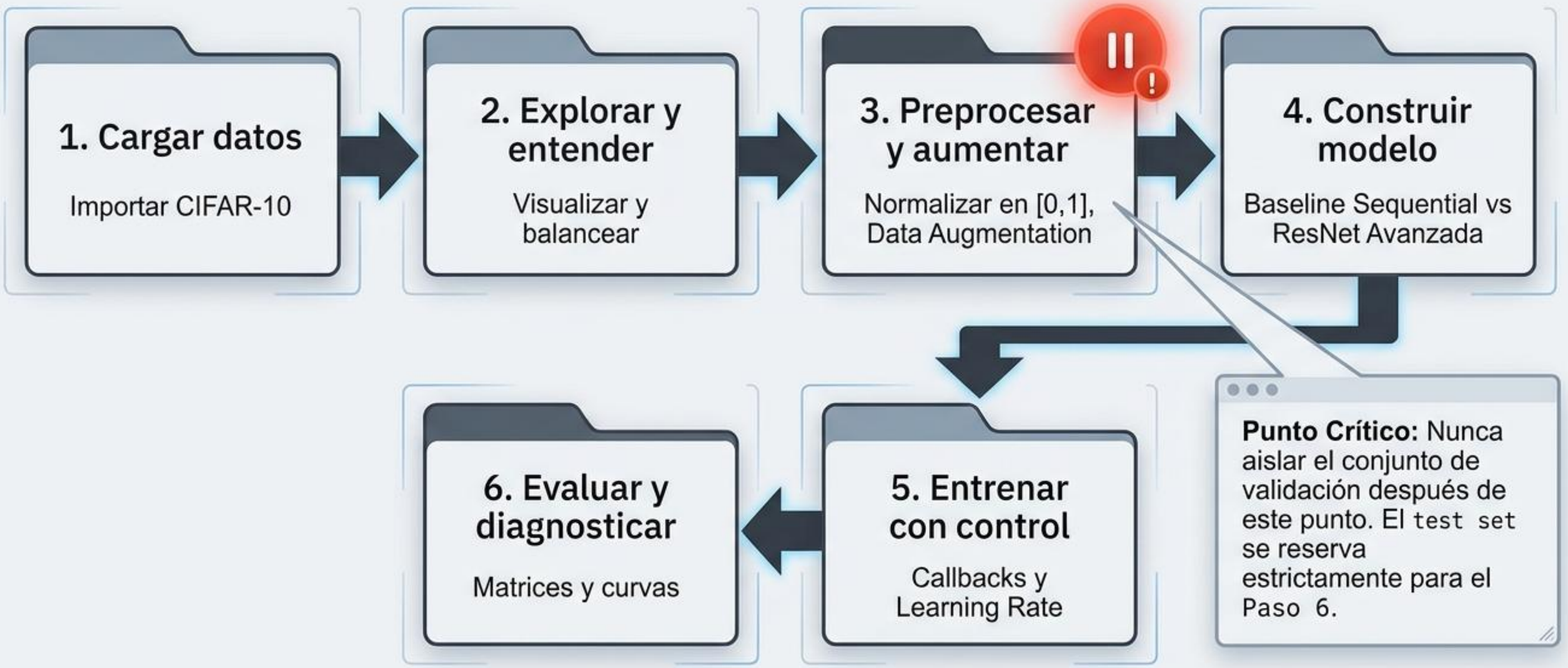
Semana 05 — Implementación Práctica de CNNs con Keras/TensorFlow

Material Complementario: De la teoría al código ejecutable

```
# Asignatura: Ingeniería del Conocimiento (ISO56B)
# Programa: Ingeniería de Sistemas — UNCP
# Docente: Msc. Jaime Antonio Huaytalla Pariona
# Audiencia: Pregrado, nivel intermedio-avanzado
```

MLOps Pipeline: Visualización del Proceso



```python
import tensorflow as tf
from sklearn.model_selection import train_test_split

# Cargar CIFAR-10
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.cifar10.load_data()

# Normalización
x_train = x_train.astype('float32') / 255.0

# Partición (Aislar validación)
x_train, x_val, y_train, y_val = train_test_split(
    x_train, y_train, test_size=0.1, stratify=y_train
)
```

¿Por qué separar validación?
Usar el conjunto de test para monitorear el entrenamiento es un error metodológico que contamina la evaluación final.

```
> Entrenando: (45000, 32, 32, 3)
> Validación: (5000, 32, 32, 3)
> Test:       (10000, 32, 32, 3)
> Rango:      [0.0, 1.0]
```



```python
model_basico = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu',
                  padding='same',input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu',
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(128, (3,3), activation='relu',
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(10, activation='softmax')
])
```

Control Espacial: En CIFAR-10 (32x32), las convoluciones sin *padding* agotarían la imagen en pocas capas. **padding='same'** mantiene la resolución.

Análisis de Tensores y Parámetros

Capa	Tensor de Salida	Parámetros
Bloque 1 (Bordes)	32x32x32	~10k
Bloque 2 (Medio Nivel)	16x16x64	~55k
Bloque 3 (Alto Nivel)	8x8x128	~221k
Clasificador	10	~527k

Total: ~814,000 parámetros

Eficiencia arquitectónica: Completamente manejable frente a los 150.5M que requeriría un Perceptrón Multicapa (MLP).

Capa de Augmentación

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomRotation(0.05),
    layers.RandomZoom((-0.1, 0.1)),
    layers.RandomTranslation(0.1, 0.1)
    layers.RandomTranslation(0.1, 0.1)
])
```

Nota: Se aplica solo durante el entrenamiento. Multiplica la diversidad de datos en tiempo de ejecución sin ocupar espacio en disco.



Original



Horizontal Flip



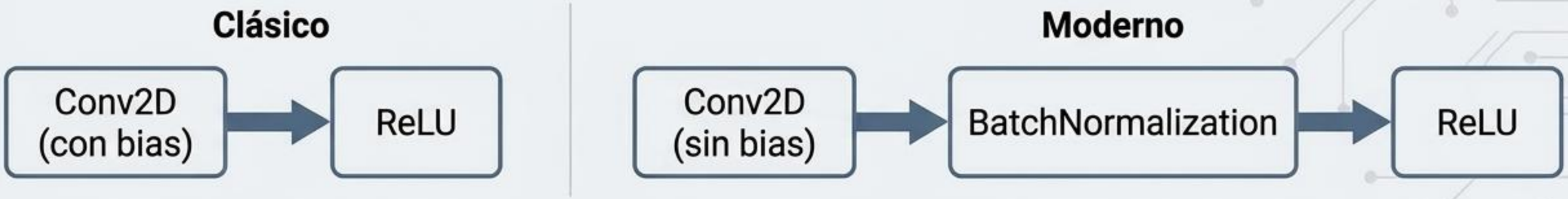
Vertical Flip



Rotación ±18°

- ✅ **Horizontal Flip:** Un auto mirando a la izquierda sigue siendo un auto.
- ❌ **Vertical Flip:** Un barco al revés no es una escena natural (Destruye la semántica).
- ✅ **Rotación (±18°):** Simula objetos ligeramente inclinados.
- ✅ **Zoom (±10%):** Simula diferentes distancias de captura.

Evolución de la Arquitectura: ConvBlock



Código Refactorizado (Moderno)

```
1: x = layers.Conv2D(32, 3, padding='same', use_bias=False,
2:                          kernel_initializer='he_normal')(x)
3: x = layers.BatchNormalization()(x)
4: x = layers.ReLU()(x)
5:
6: # Reemplazo estructural para robustez
7: x = layers.GlobalAveragePooling2D()(x)
```

Redundante: Batch Normalization aplica su propio parámetro de shift (β), haciendo el bias de la convolución matemáticamente innecesario.

Inicialización Óptima: Mantiene la varianza de los gradientes cuando se usan activaciones ReLU, evitando gradientes desvanecientes.

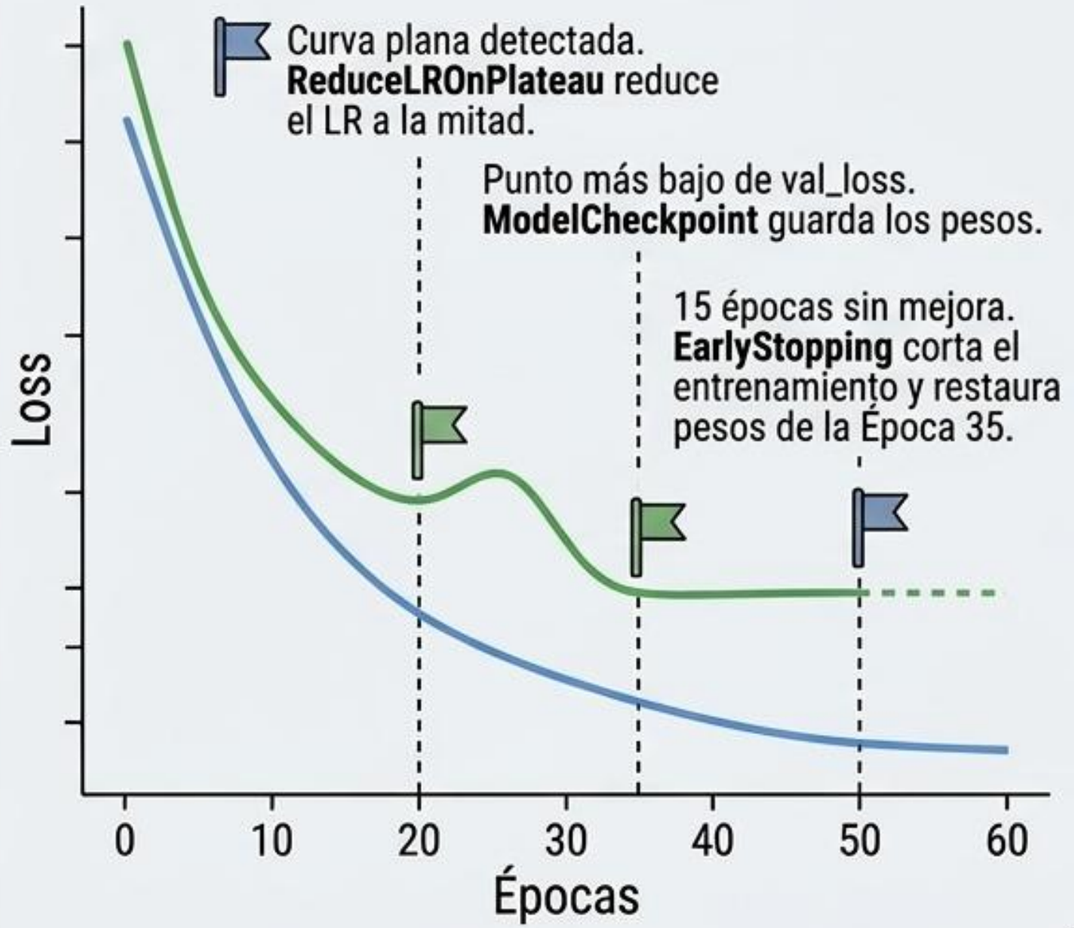
Control de Entrenamiento con Callbacks

Sentinelas Automáticos (Callbacks)

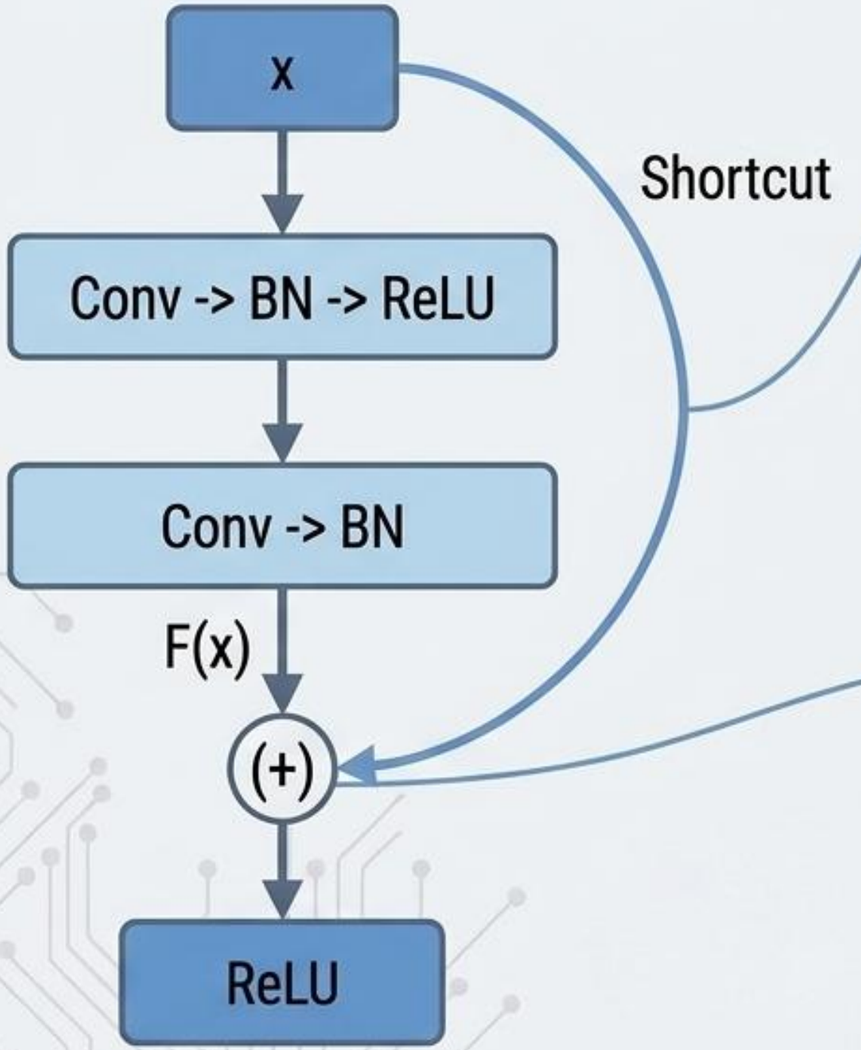
```
callbacks = [
    EarlyStopping(monitor='val_loss', patience=15,
                  restore_best_weights=True),

    ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                      patience=5),

    ModelCheckpoint('mejor.keras', save_best_only=True)
]
```



Matemática vs residual




```python
def bloque_residual(x, filtros, stride=1):
    shortcut = x

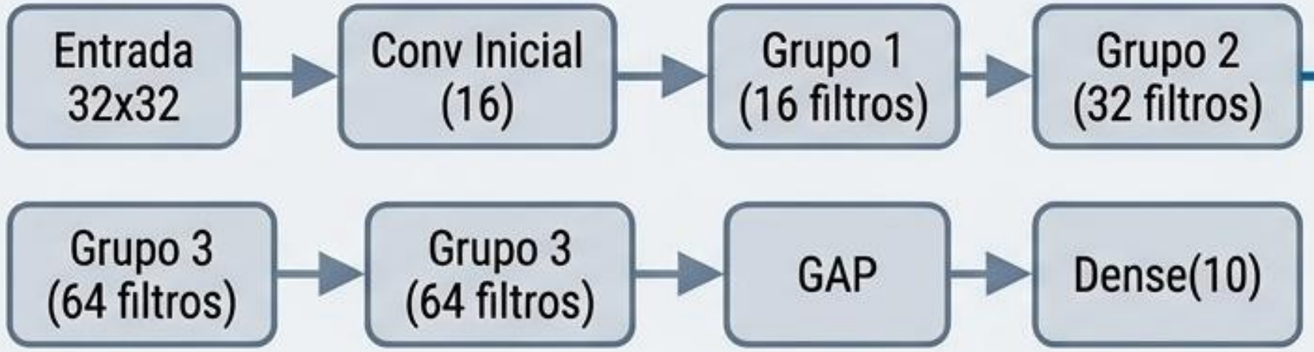
    out = layers.Conv2D(...)(x)
    out = layers.BatchNormalization()(out)
    out = layers.ReLU()(out)
    out = layers.Conv2D(...)(out)
    out = layers.BatchNormalization()(out)

    if stride != 1 or x.shape[-1] != filtros:
        shortcut = layers.Conv2D(filtros, 1, strides=stride)(x)
        shortcut = layers.BatchNormalization()(shortcut)

    out = layers.Add()([out, shortcut])
    out = layers.ReLU()(out)
    return out
```

Lógica del Atajo (Shortcut)
¿Cuándo proyectar? Si cambiamos el número de filtros o reducimos la dimensión espacial (stride=2), necesitamos aplicar una convolución 1x1 en el atajo para igualar las dimensiones antes de sumar los tensores.

Escalado de la Arquitectura ResNet: El poder del parámetro “n”



Lógica de Escalado (Loop)

```
for i in range(n):
    x = bloque_residual(x, 16, stride=1)

# Siguiente grupo reduce dimensionalidad
x = bloque_residual(x, 32, stride=2)
for i in range(n-1):
    x = bloque_residual(x, 32, stride=1)
```

Escalando la Arquitectura (El poder del parámetro 'n')

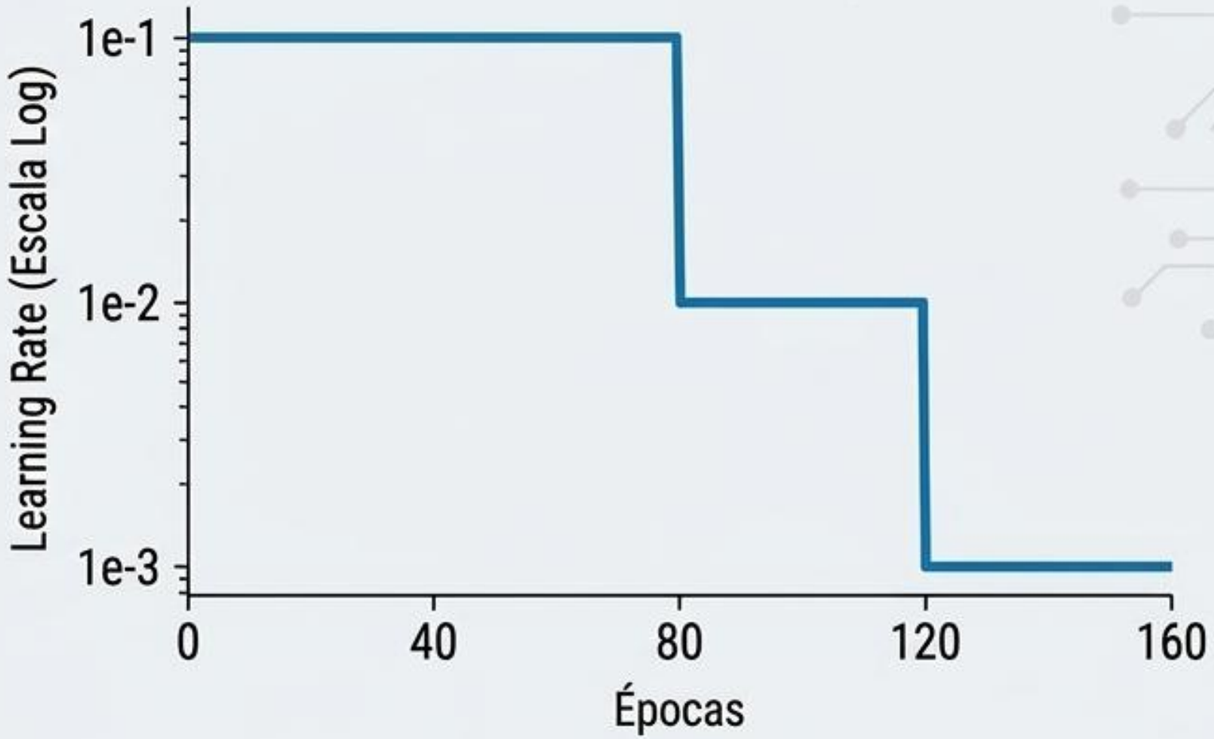
Configuración	Red Resultante	Parámetros	Accuracy Esperado (CIFAR-10)
n = 3	ResNet-20	~272K	~91.5%
n = 5	ResNet-32	~464K	~92.5%
n = 9	ResNet-56	~855K	~93.0%
n = 18	ResNet-110	~1.7M	~93.6%

Entrenamiento y Planificación del LR

Condiciones del Paper Original (He et al., 2015)

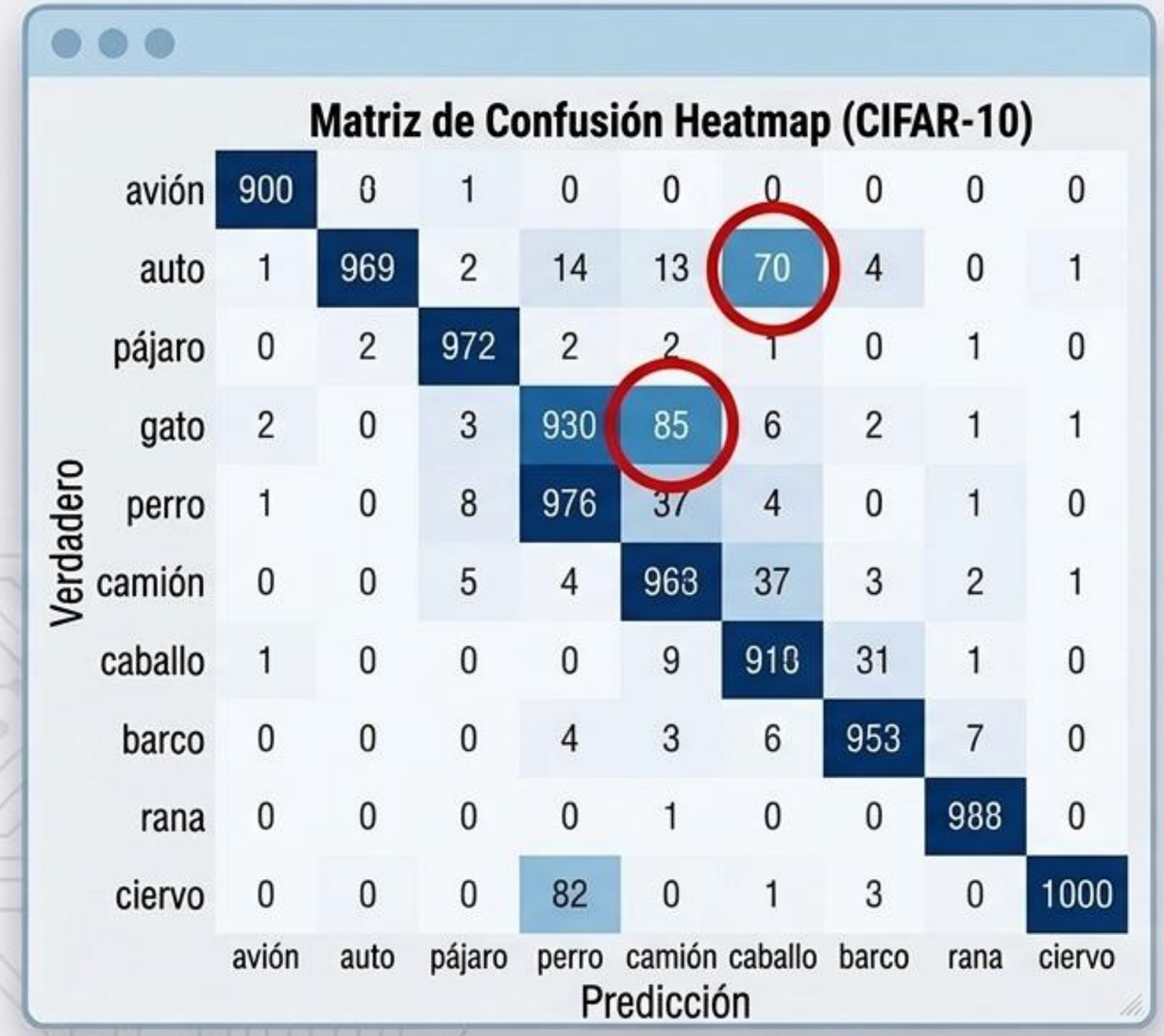
```
resnet.compile(
    optimizer=tf.keras.optimizers.SGD(
        learning_rate=0.1,
        momentum=0.9,
        weight_decay=1e-4
    ),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

def lr_schedule(epoch, lr):
    if epoch in [80, 120]:
        return lr * 0.1
    return lr
```



Decisión Arquitectónica: A diferencia de Adam (que es ideal para prototipado rápido), el paper original emplea SGD con Momentum y Regularización L2 (weight_decay) acoplado a una caída abrupta del LR para exprimir el máximo rendimiento y generalización de la ResNet.

Análisis de Errores: Matriz de Confusión y Desglose de Clases



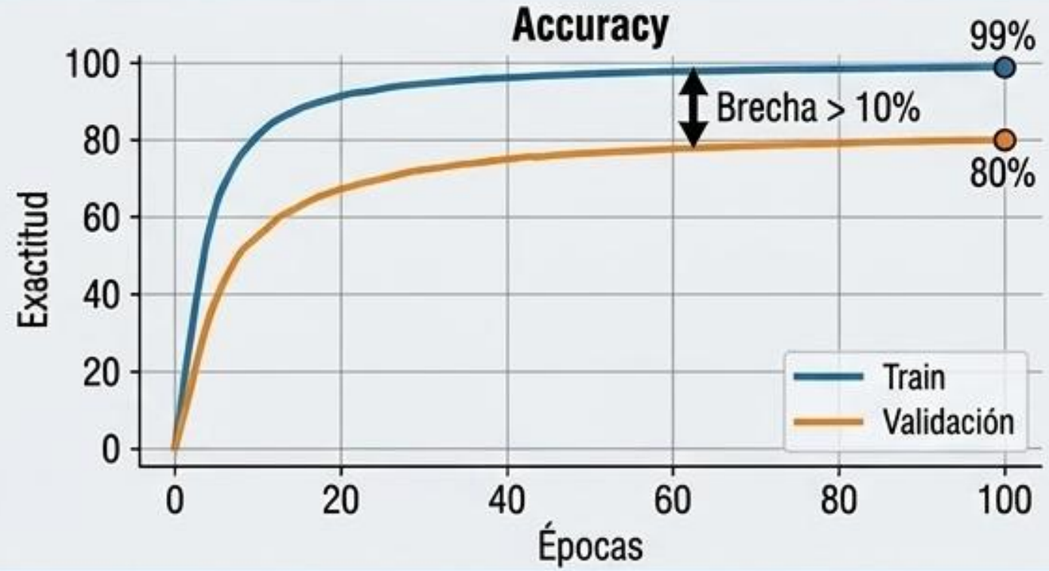

```
print(classification_report(y_true, y_pred))
```

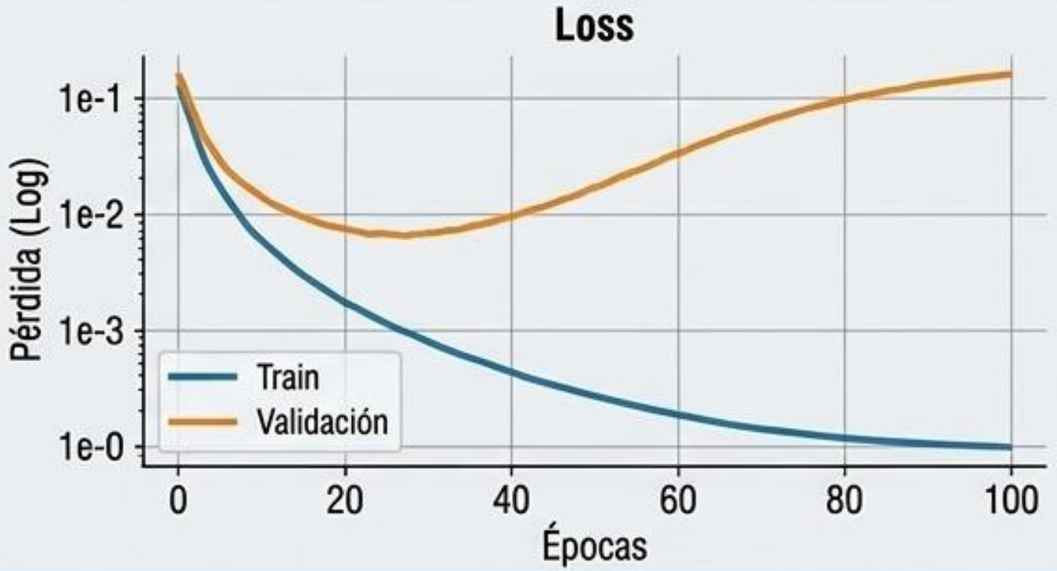
¿Qué buscar en la Matriz?

- **Diagonal Fuerte**: Indica una alta precisión de clasificación general.
- **Zonas de Confusión Críticas** (Falsos Positivos/Negativos):
  - **Gato ↔ Perro**: El modelo confunde texturas de pelaje y estructuras faciales animales similares.
  - **Auto ↔ Camión**: Confusión debido a perfiles vehiculares y bordes rectos similares.
  - **Avión ↔ Pájaro**: Ambos aparecen frecuentemente sobre un fondo de cielo azul sólido.

Conclusión: Un accuracy global del 90% puede enmascarar un F1-score desastroso en clases específicas. Siempre revisar el desglose.

Diagnóstico de Modelo: Análisis de Curvas y Árbol de Decisión





Árbol de Decisión de Diagnóstico

```
Brecha = train_acc - val_acc

IF Brecha > 10%:
    [⚠️ SÍNTOMA]: OVERFITTING (Memorización)
    [🛠️ ACCIÓN]:  Incrementar Data Augmentation, añadir capas de Dropout,
                  aumentar el weight_decay, o reducir filtros.

ELIF val_acc < 70%:
    [⚠️ SÍNTOMA]: UNDERFITTING (Falta de Capacidad)
    [🛠️ ACCIÓN]:  Modelo muy pequeño (aumentar 'n' en la ResNet),
                  entrenar por más épocas, verificar fallos de normalización.

ELSE:
    [✅ SÍNTOMA]: BUEN AJUSTE
    [🛠️ ACCIÓN]:  Guardar pesos. El modelo generaliza correctamente.
```

```
$ python check_hardware.py
> tf.config.list_physical_devices('GPU')
> ✅ GPU disponible: Tesla T4

> # Habilitando Mixed Precision para acelerar x2
> tf.keras.mixed_precision.set_global_policy('mixed_float16')
> Política activada: mixed_float16
```

Usa float16 para cálculos pesados en las convoluciones y float32 para la acumulación segura de gradientes. Duplica la velocidad en GPUs modernas (T4/A100) sin perder precisión.

Tiempos de Entrenamiento (GPU T4 - CIFAR-10)

Arquitectura	Parámetros	Tiempo por Época
CNN Básica (Sequential)	~814K	~8 segundos
CNN Mejorada (BatchNorm)	~900K	~12 segundos
ResNet-20 (He et al.)	~272K	~15 segundos
ResNet-56	~855K	~35 segundos

Errores Comunes (Gotchas)

❌ MAL	✅ BIEN
`mean = np.concatenate([train, test]).mean()`	`mean = x_train.mean()` (Estandarizar el test SOLO con stats del train para evitar Data Leakage)
`pred = model(x_test, training=True)`	`pred = model(x_test, training=False)` (training=False desactiva el Dropout y congela los promedios del BatchNorm en inferencia)

Diccionario de Traducción Final (Teoría ↔ Práctica)

Concepto Matemático / Teórico	Implementación en API Keras
Extracción Local / Weight Sharing	`layers.Conv2D()`
Bloque Residual: F(x) + x	`layers.Add()[out, shortcut])`
Normalización de Covariable Interna	`layers.BatchNormalization()` + `use_bias=False`
Reemplazo de Capas Fully Connected	`layers.GlobalAveragePooling2D()`
Mitigación de Degradación Profunda	Incrementar parámetro 'n' en `construir_resnet()`